



POLITECNICO
MILANO 1863

Politecnico di Milano

Dipartimento di Ingegneria Industriale e dell'Informazione

Corso di Laurea in Ingegneria Informatica

Prof. Fabio Salice

Anno Accademico 2023/2024

Relazione Prova Finale - Progetto di Reti Logiche

Progettazione e sintesi di un dispositivo programmabile

Emanuele Minotti - 10764444

Filippo Morelli - 10799749

Indice

1	Introduzione	1
1.1	Descrizione generale e funzionamento	1
1.2	Interfaccia del componente	1
2	Architettura	3
2.1	Segnali interni	3
2.2	Moduli	4
2.2.1	Address counter	4
2.2.2	Word counter	4
2.2.3	Credibility counter	5
2.2.4	Data register	5
2.2.5	Multiplexer	5
2.2.6	Macchina a stati	5
3	Risultati sperimentali	7
3.1	Sintesi	7
3.1.1	Report utilization	7
3.1.2	Report timing	7
3.2	Simulazioni	8
3.2.1	Sequenza vuota	8
3.2.2	Elaborazione multipla	8
3.2.3	Reset asincrono	8
3.2.4	Test completo	9
4	Conclusioni	10

1 Introduzione

1.1 Descrizione generale e funzionamento

Questo progetto ha l'obiettivo di sviluppare un modulo hardware programmabile in linguaggio VHDL per leggere e processare una sequenza di dati memorizzati in celle alterne della memoria.

Il modulo inizia l'elaborazione quando riceve un segnale di avvio e un indirizzo iniziale. La sequenza da processare ha una lunghezza specificata. Il modulo legge i dati dalla memoria e sostituisce ogni valore nullo con l'ultimo valore valido letto. Inoltre, calcola un indicatore di "credibilità" per ciascun valore, da inserire nella cella adiacente: questo indicatore inizia da 31 per ogni valore diverso da zero e diminuisce ogni volta che si incontra un valore nullo, senza mai scendere sotto zero. Se il primo valore della sequenza è nullo, esso rimane tale e il valore di credibilità associato viene impostato a zero. Lo stesso accade fino al raggiungimento del primo valore valido.

L'elaborazione termina quando il modulo ha finito di aggiornare la sequenza e i relativi valori di credibilità, segnalando la fine del processo con un segnale di completamento. Il modulo si resetta e si prepara per una nuova esecuzione del processo.

1.2 Interfaccia del componente

Di seguito vengono riportati i segnali che consentono la comunicazione tra la memoria ed il componente progettato.

- **i_clk:** sincronizza le operazioni del componente.
- **i_rst:** inizializza il componente per ricevere il primo segnale di START.
- **i_start:** avvia l'elaborazione dei dati.
- **i_k:** rappresenta la lunghezza della sequenza di dati.
- **i_addr:** rappresenta l'indirizzo di memoria da cui iniziare l'elaborazione della sequenza.
- **o_done:** indica al Test Bench la fine dell'elaborazione.
- **o_mem_addr:** contiene l'indirizzo di memoria al quale leggere/scrivere il dato.
- **i_mem_data:** contiene il dato letto all'indirizzo specificato.
- **o_mem_data:** contiene il dato da scrivere all'indirizzo specificato.
- **o_mem_en:** abilita la comunicazione con la memoria.
- **o_mem_we:** abilita la scrittura in memoria.

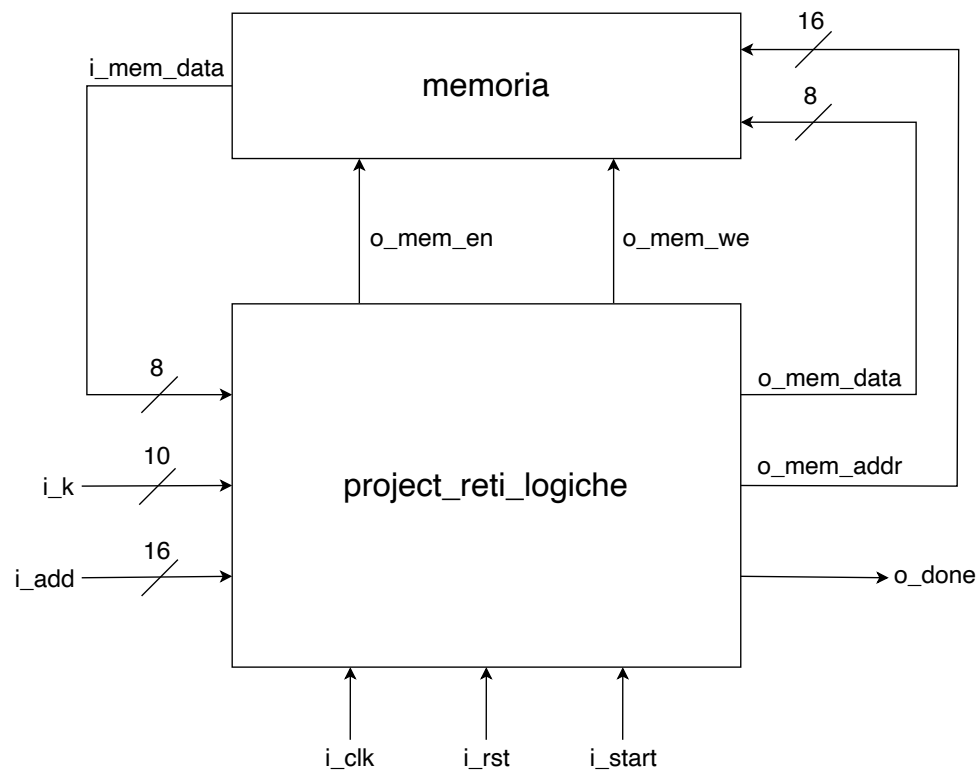


Figura 1: Interfaccia del componente

2 Architettura

Per affrontare il problema, abbiamo adottato un approccio modulare, scomponendo il sistema in componenti distinti, ognuno dei quali è implementato come un processo separato nel codice VHDL del componente. Questa suddivisione consente di isolare e gestire singolarmente le diverse responsabilità funzionali del sistema, migliorando la leggibilità e la manutenibilità del codice. L'architettura complessiva comprende cinque moduli principali che collaborano per elaborare correttamente la sequenza di dati.

L'approccio modulare è stato scelto per diversi motivi validi. In primo luogo, facilita la verifica e il debug del sistema, permettendo di testare e validare ciascun modulo in modo indipendente. In secondo luogo, promuove la riusabilità del codice, poiché moduli ben definiti possono essere riutilizzati in altri progetti o contesti. Infine, questa architettura consente una maggiore flessibilità e scalabilità, rendendo più semplice l'implementazione di modifiche o l'aggiunta di nuove funzionalità senza compromettere l'intero sistema.

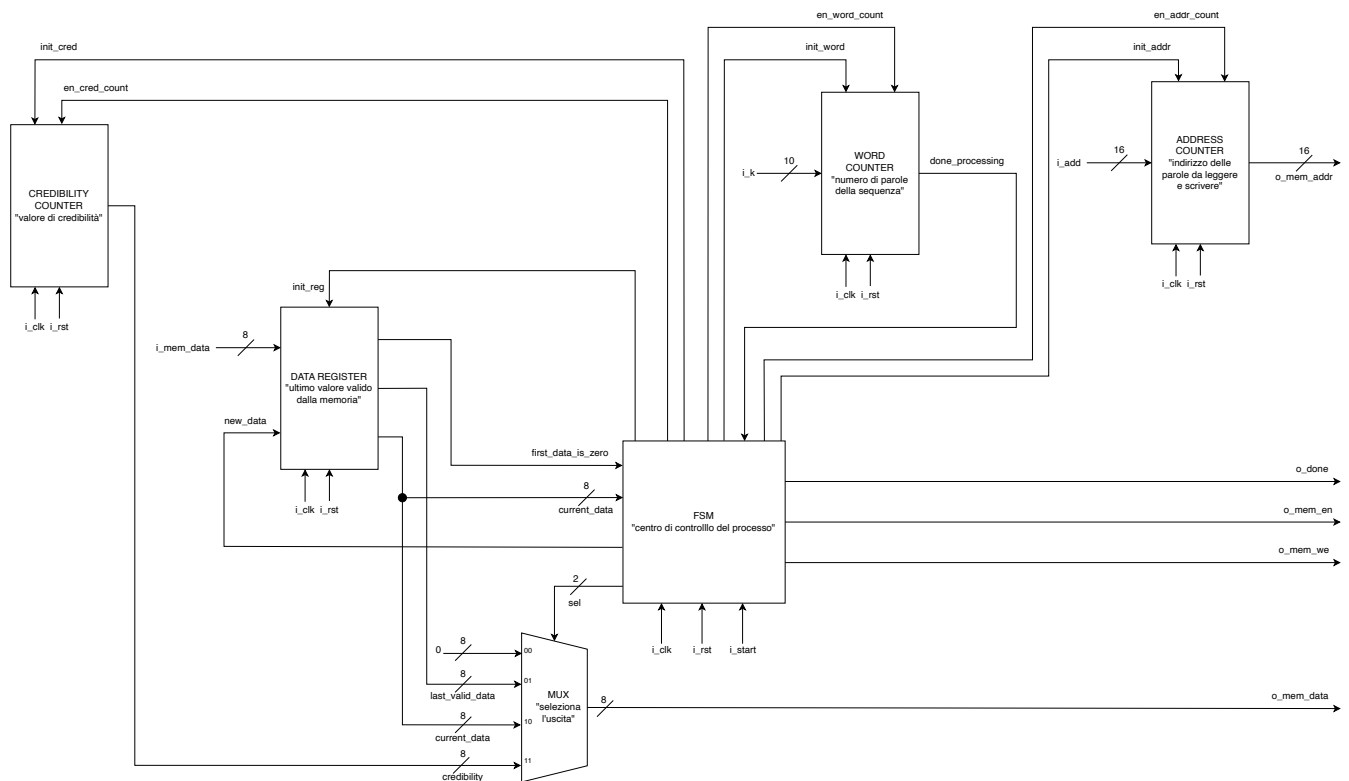


Figura 2: Schema a blocchi del componente

2.1 Segnali interni

Di seguito una breve descrizione di tutti i segnali interni utilizzati per implementare le funzionalità del componente.

- **first_data_is_zero:** flag utilizzato per la gestione del caso particolare in cui il primo valore è zero.
- **curr_addr:** memorizza l'indirizzo corrente al quale leggere/scrivere i dati in memoria.
- **credibility:** rappresenta il valore di credibilità corrente.
- **word_count:** rappresenta il numero di parole lette dalla memoria.
- **en_addr_count:** abilita il modulo che si occupa di calcolare il corretto indirizzo di memoria.
- **en_word_count:** abilita il modulo contatore di parole.
- **en_cred_count:** abilita il modulo che si occupa di calcolare il valore di credibilità per la parola corrente.
- **init_addr:** reinizializza il modulo Address counter.
- **init_word:** reinizializza il modulo Word counter.
- **init_cred:** reinizializza il modulo Credibility counter.
- **init_reg:** reinizializza il modulo Data register.
- **sel:** indica quale valore collegare all'uscita.
- **last_valid_data:** mantiene l'ultimo valore valido letto dalla memoria.
- **current_data:** memorizza il valore corrente appena letto dalla memoria.
- **new_data:** comunica che un nuovo valore valido è stato letto dalla memoria.
- **done_processing:** comunica il termine della computazione.

2.2 Moduli

2.2.1 Address counter

Calcola l'indirizzo della cella di memoria del dato da leggere o scrivere. Ad inizio computazione 'curr_addr' viene posto uguale a 'i_addr'. Successivamente viene incrementato di un'unità sul fronte di salita del clock se il segnale di 'en_addr_count' è attivo, mentre in presenza di un segnale di reset 'curr_addr' viene riportato a zero.

2.2.2 Word counter

Conta fino al numero di parole K . Il conteggio parziale, inizialmente posto a zero, viene incrementato di un'unità ogni volta che viene letta una parola dalla memoria. Una volta raggiunto il valore K fornito dal testbench, il segnale 'done_processing' viene attivato,

indicando la conclusione della computazione. In presenza di un segnale di reset sia 'word_count' che 'done_processing' vengono riportati a zero.

2.2.3 Credibility counter

Calcola il valore di credibilità che viene scritto in memoria durante la computazione. Partendo dal valore 31 di default, esso rimane tale se viene letta una parola diversa da zero, mentre viene decrementato di un'unità (senza mai scendere al di sotto di zero) per ogni dato non valido consecutivo letto. In presenza di un segnale di reset, 'credibility' torna al suo valore di default.

2.2.4 Data register

Aggiorna tutti i segnali associati alla lettura del dato corrente. In particolare, ad ogni ciclo di clock, se non c'è stata nessuna reinizializzazione, il segnale 'current_data' viene aggiornato con il valore di 'i_mem_data'. Inoltre, se 'new_data' è uguale a 1, 'first_data_is_zero' viene impostato a zero e 'last_valid_data' viene aggiornato con il valore di 'i_mem_data'.

2.2.5 Multiplexer

In base al valore del segnale di selezione, l'uscita 'o_mem_data' viene collegata al corrispondente dato da scrivere in memoria.

2.2.6 Macchina a stati

L'ultimo componente del modulo progettato è la macchina a stati finiti, che ha il compito di gestire l'intero processo di elaborazione della sequenza in ingresso. La FSM coordina le operazioni degli altri componenti, assicurando che ogni fase dell'elaborazione avvenga nell'ordine corretto e nel momento opportuno.

Di seguito vengono elencati gli stati della macchina con le relative uscite. Gli stati S2, S3 ed S5 sono considerati stati *idle* e non producono alcuna uscita. Tuttavia, si è rivelato necessario aggiungerli in fase di progettazione per garantire la corretta sincronizzazione dei moduli tra loro e con il clock.

- S0: Questo è lo stato iniziale della FSM. La transizione verso S1 avviene quando il segnale 'i_rst' viene abbassato.
- S1: Stato di setup in cui tutti i moduli vengono reinizializzati, preparandoli per una nuova computazione non appena il segnale 'i_start' viene alzato. Viene abilitata la comunicazione con la memoria.
- S2: Stato di attesa dell'arrivo di una nuova parola, viene attraversato ciclicamente per scorrere l'intera sequenza di dati. Una volta che 'done_processing' viene alzato, avviene la transizione verso lo stato S7.

- **S4:** In questo stato, i segnali vengono gestiti in base al valore letto dalla memoria. Se il primo valore della sequenza è nullo, il segnale di selezione viene modificato per portare un valore nullo in uscita. Se invece è un valore nullo in mezzo alla sequenza, il segnale per attivare *Credibility counter* viene alzato e la selezione cambia per portare in uscita l'ultimo valore valido letto. Se il valore letto è diverso da zero, *Credibility counter* viene reinizializzato, il segnale 'new_data' viene alzato e la selezione viene modificata per portare in uscita il dato corrente. Indipendentemente dal valore letto, viene abilitata la scrittura in memoria e attivato *Address counter*.
- **S6:** Stato che modifica la selezione per la scrittura della credibilità. Viene scritto 0 se non è mai stato letto un valore non nullo, altrimenti viene scritto il valore contenuto in *Credibility*. Inoltre, i segnali per l'abilitazione della scrittura e dei moduli *Address counter* e *Word counter* vengono alzati.
- **S7:** Stato di attesa che aspetta che il segnale 'i_start' venga abbassato, permettendo così di tornare allo stato di setup S1.

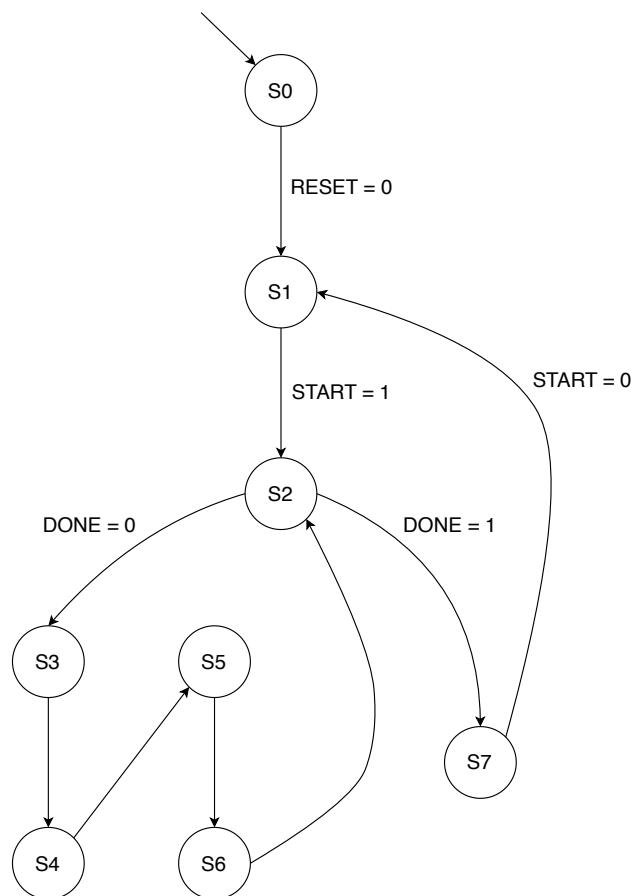


Figura 3: Macchina a stati finiti

3 Risultati sperimentali

3.1 Sintesi

3.1.1 Report utilization

Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs*	63	0	0	78600	0.08
LUT as Logic	63	0	0	78600	0.08
LUT as Memory	0	0	0	26600	0.00
Slice Registers	57	0	0	157200	0.04
Register as Flip Flop	57	0	0	157200	0.04
Register as Latch	0	0	0	157200	0.00
F7 Muxes	0	0	0	39300	0.00
F8 Muxes	0	0	0	19650	0.00

Solo una minima parte delle *Lookup Table* e dei registri disponibili sulla scheda

Artix – 7FPGAXc7a200t fbg484 – 1 utilizzata in fase di sintesi è stata necessaria per implementare il componente progettato.

Inoltre, nessuno dei registri utilizzati è del tipo *Latch*, condizione necessaria a garantire l'assenza di ritardi di propagazione indesiderati.

3.1.2 Report timing

Slack (MET) : 17.369ns (required time - arrival time)

Source: current_data_reg[3]/C
(rising edge-triggered cell FDCE clocked by clock
{rise@0.000ns fall@5.000ns period=20.000ns})

Destination: credibility_reg[0]/CE
(rising edge-triggered cell FDPE clocked by clock
{rise@0.000ns fall@5.000ns period=20.000ns})

Path Group: clock

Path Type: Setup (Max at Slow Process Corner)

Requirement: 20.000ns (clock rise@20.000ns - clock rise@0.000ns)

Data Path Delay: 2.210ns (logic 0.540ns (24.434%) route 1.670ns (75.566%))

Logic Levels: 3 (LUT4=1 LUT5=1 LUT6=1)

Il progetto deve funzionare con un periodo di clock di almeno 20 ns, e questo requisito è stato verificato e rispettato come mostrato nel report di timing fornito dall'ambiente di sviluppo. Il tempo di percorso dati (data path delay) è di soli 2.210 ns, suddiviso tra logica (0.540 ns) e routing (1.670 ns).

Questo ampio margine di slack conferma che il progetto è in grado di operare correttamente con il periodo di clock specificato, garantendo la stabilità e l'affidabilità del dispositivo anche nelle condizioni di processo più lente.

3.2 Simulazioni

Abbiamo testato il comportamento del nostro dispositivo in vari casi limite, conformi e coerenti con le specifiche, che abbiamo ritenuto rilevanti per assicurarci del suo corretto funzionamento.

Durante le simulazioni, abbiamo assunto che tutti i dati forniti dal test bench e dalla memoria fossero sempre nel formato desiderato e privi di errori fondamentali. Di conseguenza, non abbiamo implementato controlli aggiuntivi sugli input, concentrandoci invece sull'elaborazione e la gestione efficiente dei dati forniti.

3.2.1 Sequenza vuota

Abbiamo verificato il comportamento del nostro dispositivo con una sequenza vuota, ossia una sequenza con lunghezza $K = 0$. In questo scenario, il modulo dovrebbe immediatamente segnalare il completamento dell'elaborazione tramite il segnale *DONE* senza effettuare alcuna operazione di lettura o scrittura in memoria. Questo test garantisce che il dispositivo gestisca correttamente il caso limite di una sequenza priva di dati, evitando operazioni inutili.

3.2.2 Elaborazione multipla

Per testare la capacità del dispositivo di gestire più elaborazioni consecutive, abbiamo eseguito una serie di cicli di avvio (*START*) e completamento (*DONE*). Ogni ciclo ha comportato l'elaborazione di una nuova sequenza di dati con diverse lunghezze e indirizzi di partenza. Questo test ha confermato che il dispositivo può reinizializzarsi correttamente tra un'elaborazione e l'altra e che tutte le operazioni vengono eseguite senza errori o conflitti.

3.2.3 Reset asincrono

Abbiamo introdotto il segnale di reset asincrono (*RESET*) durante diverse fasi dell'elaborazione per assicurare che il dispositivo si re-inizializzi correttamente. Questo test ha dimostrato che il modulo è in grado di interrompere l'elaborazione corrente e tornare allo stato

iniziale in modo affidabile, privo di malfunzionamenti, e pronto per ricevere un nuovo segnale di *START*.

3.2.4 Test completo

Il test completo ha coinvolto l'elaborazione di una sequenza di dati complessa, includendo diversi casi limite come la presenza di valori zero all'inizio, in mezzo e alla fine della sequenza. Abbiamo verificato che il modulo sostituisse correttamente i valori zero con l'ultimo valore valido e calcolasse i valori di credibilità in modo accurato. Questo test ha confermato il corretto funzionamento del dispositivo in un ambiente realistico e complesso, assicurandone la robustezza e l'affidabilità.

4 Conclusioni

Il progetto offre l'opportunità di sviluppare competenze nell'ambito della progettazione di hardware dedicato all'elaborazione e alla gestione efficiente dei dati.

Attraverso questo lavoro, è stato possibile acquisire una più profonda comprensione delle metodologie e delle tecniche necessarie per implementare sistemi sincroni nel campo dell'ingegneria hardware.

La capacità di tradurre concetti teorici in soluzioni pratiche e funzionali è un risultato significativo di questo lavoro, dimostrando la nostra preparazione nell'affrontare sfide complesse.